

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE DE SISTEMAS

ENGENHARIA DE SOFTWARE - II

GRUPOS DE ENGENHARIA DE SOFTWARE

Fatec Campinas

- **O que é Engenharia de Software?**
  - Engenharia de Software é sobre criar software de forma **sistemática, disciplinada e quantificável**.
  - Não é só 'codar'. É sobre gerenciar complexidade, garantir qualidade, atender prazos e orçamentos.

GRUPOS DE ENGENHARIA DE SOFTWARE

Fatec Campinas

- **A Necessidade do Trabalho em Grupo:**
  - Quase nenhum software complexo é construído por uma única pessoa. O trabalho em equipe é a **regra**, não a exceção. Isso acontece porque os projetos são grandes, demandam diversas habilidades e diferentes perspectivas.
  - Trabalhar em grupo melhora a qualidade (pense na revisão de código, onde um colega encontra um erro que você não viu), distribui a carga de trabalho e permite aprender muito com os colegas. É como uma orquestra, cada músico tem seu instrumento, mas a melodia só sai perfeita com todos juntos.
  - **Exemplo Prático:** Imagine a equipe que desenvolve o sistema de e-commerce da **Amazon**. Não é uma pessoa só. Há desenvolvedores de front-end (o que você vê), back-end (o que faz tudo funcionar por trás), especialistas em banco de dados, designers de UX (experiência do usuário), testadores, gerentes de produto. Cada um com sua especialidade, contribuindo para um produto gigantesco e funcional. Isso só é possível com um trabalho em equipe muito bem orquestrado.

GRUPOS DE ENGENHARIA DE SOFTWARE

Fatec Campinas

- **Formação de Equipes (Pontos a Ponderar):**
    - Como formar bons grupos? Não é só juntar pessoas aleatoriamente, principalmente em projetos complexos.
    - **Tamanho:** Grupos pequenos (3 a 5 pessoas) tendem a ser mais eficientes em projetos iniciais. Grupos grandes podem gerar mais desafios de comunicação e coordenação.
    - **Diversidade de Habilidades:** Idealmente, o grupo deve ter diferentes perfis: alguém mais forte em design, outro em lógica de programação, outro em comunicação com o cliente. Essa diversidade enriquece o projeto.
        - **Exemplo Prático:** Se um grupo só tem programadores back-end, pode ter dificuldade em criar uma interface de usuário atraente. Se só tem designers, pode não conseguir implementar a lógica complexa por trás. Um grupo equilibrado é fundamental para cobrir todas as áreas necessárias do projeto.

GRUPOS DE ENGENHARIA DE SOFTWARE

Fatec Campinas

- **Interesses Comuns e Boa Dinâmica:**
  - Se os membros têm algum interesse comum no tema do projeto, a motivação aumenta. Além disso, é importante considerar a compatibilidade e a capacidade de colaborar. No mundo real, você não escolhe 100% seus colegas, mas desenvolver habilidades para trabalhar com diferentes perfis é parte da Engenharia de Software.

DEFINIÇÃO DO PROJETO

- Antes mesmo de pensar em quem vai fazer o quê, precisamos ter uma ideia clara do **que** será o projeto. Uma **boa definição do projeto** é a chave para o sucesso.

  - **Origem da Ideia:** Uma ideia de projeto pode vir de uma necessidade real (um problema que precisa ser resolvido), de um cliente, de uma inovação de mercado ou até mesmo de um desafio acadêmico.

  - **Problema a ser Resolvido:** Qual é o problema que nosso software vai solucionar?
    - É fundamental ter clareza desde o início. Sem um problema claro, o software não terá valor real para ninguém.
      - **Exemplo Prático:** Se você vai criar um aplicativo, qual dor ele alivia?
        - O **Waze** resolve o problema do trânsito.
        - O **WhatsApp** resolve o problema da comunicação rápida e barata.
        - O **Google Maps** resolve o problema de encontrar lugares e rotas.
  - Um bom projeto sempre começa com a identificação de um problema que vale a pena ser resolvido.

DEFINIÇÃO DO PROJETO

- **Objetivos do Projeto:** O que queremos alcançar com este software?
  - Sejam específicos! Por exemplo: 'Automatizar o controle de estoque de uma pequena loja para reduzir perdas em 20%', ou 'Melhorar a comunicação entre alunos e professores em 50%'. Isso dá direção ao trabalho.

- **Escopo Inicial:** O que o software *vai* e *não vai* fazer?
  - É um primeiro rascunho para não nos perdermos. O escopo inicial ajuda a gerenciar expectativas e evitar que o projeto se torne gigante e incontrolável.

    - **Exemplo Prático:** Se a ideia é criar um sistema de gerenciamento de biblioteca, um escopo inicial pode ser: 'O sistema permitirá cadastrar livros e usuários, e registrar empréstimos e devoluções.' Mas ele *não vai* (nesta fase) lidar com multas, compras de livros ou integração com sistemas financeiros. Essa clareza evita que o projeto se estenda indefinidamente.

DEFINIÇÃO DO PROJETO

- **Público-Alvo:** Para quem estamos construindo isso?
  - Entender o usuário final é crucial para criar algo que seja realmente útil e fácil de usar. Se o público-alvo são idosos, a interface precisa ser simples e com letras grandes. Se são adolescentes, talvez um design mais moderno e recursos de gamificação.

DEFINIÇÃO DO PROJETO

Fatec Campinas

- Pensem em um problema que vocês ou pessoas que conhecem enfrentam no dia a dia.
- Se vocês pudessem criar um software para resolver esse problema, qual seria ele? E qual seria o objetivo principal desse software?

COMUNICAÇÃO EFETIVA

- A falha de comunicação é, talvez, a **maior causa de problemas** em projetos de software, Se a equipe não se comunica clara e constantemente, surgem mal-entendidos, retrabalho e frustração.
- Isso inclui comunicar-se com os colegas, com o cliente e com outros stakeholders. É preciso ser um bom ouvinte e um bom transmissor de ideias.

- **Ferramentas e Hábitos:** Para isso, usamos ferramentas como Slack, Microsoft Teams, e-mail e, principalmente, **reuniões regulares** (podem ser diárias e rápidas, como nas metodologias ágeis, ou semanais). O importante é que todos saibam o que está acontecendo e quais são as próximas etapas.

    - **Exemplo Prático:** Em uma equipe, se o desenvolvedor do front-end não conversa com o do back-end sobre como os dados serão enviados, eles podem construir partes que não se encaixam. O resultado? Horas de retrabalho para ajustar a comunicação entre os sistemas. A comunicação é a 'cola' do projeto.

PAPÉIS E RESPONSABILIDADES

- Quem faz o quê?
  - Se não for claro, pode haver retrabalho (duas pessoas fazendo a mesma coisa), tarefas esquecidas (ninguém fez algo que precisava) ou sobrecarga para alguns. No início do projeto, definam quem será responsável por cada área.“
  - **Papéis Comuns:** Mesmo em grupos pequenos, ter um 'líder de projeto' (para organizar reuniões e tarefas), um 'analista de requisitos' (foco em entender as necessidades), 'desenvolvedores', 'testadores’, são usados para organizar o trabalho.“
    - **Exemplo Prático:** Em um projeto de aplicativo, se ninguém é claramente responsável pelo design da interface, o aplicativo pode ficar feio e difícil de usar. Ou, se dois desenvolvedores começam a trabalhar na mesma funcionalidade sem saber, eles desperdiçam tempo e energia, e ainda podem criar conflitos no código.

GESTÃO DE CONFLITOS

- Desentendimentos são normais em qualquer grupo.
    - A chave é saber gerenciá-los de forma **construtiva**.
    - Focar no problema, não na pessoa.
    - Buscar o consenso e a melhor solução para o projeto.
- Conflitos não resolvidos podem destruir a moral da equipe e o andamento do projeto.
    - É importante que os membros se sintam à vontade para discutir abertamente, mas sempre com respeito.

PLANEJAMENTO E CRONOGRAMA

- Um projeto sem planejamento é um barco à deriva, sem saber para onde vai.

  - Precisamos de um **cronograma** com marcos importantes e prazos.
  - Mesmo em processos ágeis, que são mais flexíveis, existe planejamento para os ciclos curtos (sprints).

  - **Ferramentas Simples:** Podemos usar ferramentas simples como Trello, Asana, ou até uma planilha compartilhada para listar tarefas, definir quem é responsável e quando elas devem ser entregues. Isso dá visibilidade e ajuda a controlar o progresso.

    - **Exemplo Prático:** Imagine que um grupo tem um prazo final para entregar um sistema. Se não há um cronograma, as tarefas podem ser feitas sem ordem, deixando as mais difíceis para o final, resultando em correria e um produto incompleto ou cheio de erros. Um cronograma ajuda a distribuir o trabalho de forma inteligente ao longo do tempo.

VERSIONAMENTO DE CÓDIGO

Fatec Campinas

- Quando várias pessoas escrevem código para o mesmo projeto, como garantimos que ninguém perca o trabalho do outro ou sobrescreva arquivos?
  - É aí que entram os **Sistemas de Controle de Versão (VCS)**, essenciais na Engenharia de Software moderna.

  - O **Git** (e plataformas como GitHub ou GitLab) é a ferramenta padrão de mercado. Ele permite que todos trabalhem no mesmo código ao mesmo tempo, acompanhem as mudanças feitas por cada um, resolvam conflitos (quando duas pessoas alteram a mesma linha de código) e, o mais importante, voltem a versões anteriores se algo der errado.

    - **Exemplo Prático:** Pensem no Google Docs. Várias pessoas podem editar o mesmo documento e você vê as alterações de cada um, e pode até voltar para uma versão anterior. O Git faz isso para código, de forma muito mais poderosa e complexa. Sem ele, a colaboração em código seria caótica e quase impossível.

TESTES E QUALIDADE

- Qualidade não é algo que se adiciona no final.
    - Ela é construída ao longo de todo o processo. Testes devem ser feitos continuamente, não apenas antes da entrega final.
    - A equipe toda é responsável pela qualidade do produto. Desenvolvedores testam o que codificam, testadores verificam as funcionalidades, e até o cliente pode participar dos testes.
        - **Exemplo Prático:** Se um aplicativo de banco não é testado rigorosamente, pode haver erros nas transações, causando prejuízos financeiros aos usuários e à instituição. A qualidade é a confiança que o usuário deposita no seu software.

DOCUMENTAÇÃO

- Registrar decisões, requisitos, design do sistema, e até mesmo problemas e soluções, é vital.
    - A documentação ajuda novos membros a entenderem o projeto, previne esquecimentos e serve como base para a manutenção futura do software.
    - Não precisa ser uma pilha de papéis. Pode ser um documento online, um wiki ou até comentários bem feitos no código. O importante é que a informação esteja acessível.
        - **Exemplo Prático:** Imagine que um desenvolvedor sai da equipe no meio do projeto. Se não há documentação de como certas partes do sistema funcionam ou quais decisões foram tomadas, o próximo desenvolvedor terá que 'descobrir' tudo do zero, perdendo muito tempo e correndo o risco de introduzir novos erros.

GRUPOS ENG. SOFTWARE - EXERCICIOS

A Engenharia de Software é definida no material como uma abordagem que vai além de "codar" e se baseia em princípios fundamentais. Qual tríade de atributos descreve de forma mais rigorosa a natureza sistemática e metódica da Engenharia de Software, de acordo com o texto?

A. Adaptativa, contínua e orientada a valor, focando na entrega de código limpo.
B. Colaborativa, iterativa e flexível, priorizando a gestão da complexidade.
C. Sistemática, disciplinada e quantificável, essencial para gerenciar a complexidade e a qualidade.
D. Integrada, automatizada e escalável, garantindo o atendimento a prazos e orçamentos.
E. Funcional, robusta e eficiente, visando a melhor experiência do usuário (UX).

C. Sistemática, disciplinada e quantificável, essencial para gerenciar a complexidade e a qualidade.

GRUPOS ENG. SOFTWARE - EXERCICIOS

O trabalho em equipe é considerado a regra, e não a exceção, em Engenharia de Software. Qual é a principal combinação de fatores intrínsecos aos projetos complexos que torna a atuação de uma única pessoa quase inviável, conforme o texto?

A. A necessidade de revisão de código em tempo real e a distribuição igualitária da carga de trabalho entre os membros.
B. O imperativo de garantir a compatibilidade entre Front-End e Back-End, exigindo diferentes especialistas em arquitetura de dados.
C. A alta exigência de documentação formal e a gestão de múltiplos Stakeholders com interesses divergentes.
D. O grande tamanho dos projetos, que demandam simultaneamente uma ampla diversidade de habilidades e diferentes perspectivas.
E. A complexidade do Versionamento de Código (VCS) e a necessidade de resolver conflitos de código com múltiplas alterações.

D. O grande tamanho dos projetos, que demandam simultaneamente uma ampla diversidade de habilidades e diferentes perspectivas

GRUPOS ENG. SOFTWARE - EXERCICIOS

A diversidade de habilidades enriquece o projeto. Um exemplo prático utilizado para ilustrar a falha na diversidade é um grupo composto unicamente por programadores Back-End. Qual é a dificuldade prática que esse grupo homogêneo, focado apenas na lógica, enfrentaria?

A. Dificuldade na resolução de conflitos de código (VCS), pois a mentalidade é demasiadamente similar.
B. Dificuldade em estabelecer um Cronograma realista, por não haver um Líder de Projeto formal com visão macro.
C. Dificuldade em criar uma interface de usuário atraente, negligenciando a parte visual e a Experiência do Usuário (UX).
D. Dificuldade em definir a Origem da Ideia e o Problema a ser Resolvido, por falta de foco no cliente (Stakeholder).
E. Dificuldade em garantir a segurança dos dados e o acesso eficiente ao Banco de Dados, pela sobrecarga de foco na lógica de negócio.

C. Dificuldade em criar uma interface de usuário atraente, negligenciando a parte visual e a Experiência do Usuário (UX).

GRUPOS ENG. SOFTWARE - EXERCICIOS

Na etapa de Definição do Projeto, qual é o elemento considerado o mais fundamental para o sucesso e que, em sua ausência ou falta de clareza, impede o software de obter "valor real para ninguém"?

A. Público-Alvo, pois ele guia as decisões de usabilidade (UX) e design da interface.
B. O Escopo Inicial, pois ele evita o excesso de funcionalidades e gerencia as expectativas do Stakeholder.
C.. A Origem da Ideia, que deve preferencialmente ser uma necessidade real do mercado e não apenas um desafio acadêmico
D. Os Objetivos do Projeto, pois a falta de metas específicas e mensuráveis impede que o trabalho tenha direção.
E..O Problema a ser Resolvido, cuja clareza é fundamental para conferir valor real à solução de software

E..O Problema a ser Resolvido, cuja clareza é fundamental para conferir valor real à solução de software

GRUPOS ENG. SOFTWARE - EXERCICIOS

A falha de comunicação é apontada como a maior causa de problemas em projetos. Qual é a tríade de consequências negativas diretas que surgem quando a equipe falha em se comunicar de forma clara e constante?

A. Dificuldade na Gestão de Conflitos, atraso no Cronograma e sobrecarga do Líder de Projeto.
B. Mal-entendidos, necessidade de retrabalho e frustração entre os membros da equipe.
C. Perda de rastreabilidade de requisitos, omissão da Documentação e má qualidade do código Back-End.
D. Incompatibilidade entre Front-End e Back-End, falha de segurança e perda de confiança do Stakeholder.
E. Uso incorreto do Sistema de Controle de Versão (VCS), desperdício de tempo e energia, e Objetivos do Projeto irrealistas.

B. Mal-entendidos, necessidade de retrabalho e frustração entre os membros da equipe.

GRUPOS ENG. SOFTWARE - EXERCICIOS

A definição clara de papéis e responsabilidades ("Quem faz o quê?") é uma prática essencial para evitar quais dois problemas principais de execução em um projeto de software, que afetam diretamente a produtividade?

A. Falha de segurança e omissão da Documentação de design do sistema.
B. Retrabalho (duas pessoas fazendo a mesma coisa) e tarefas esquecidas (ninguém fez o que precisava).
C. Dificuldade na Gestão de Conflitos e má comunicação entre os desenvolvedores de Front-End e Back-End.
D. Sobrecarga do Líder de Projeto e dificuldade em definir Objetivos do Projeto mensuráveis.
E. Desvio do Escopo Inicial e introdução de bugs no código-fonte por desenvolvedores inexperientes.

B. Retrabalho (duas pessoas fazendo a mesma coisa) e tarefas esquecidas (ninguém fez o que precisava).

GRUPOS ENG. SOFTWARE - EXERCICIOS

O planejamento e o Cronograma são descritos como o "mecanismo" que impede o projeto de ser um "barco à deriva". Qual é o papel essencial do Cronograma, além de definir prazos, e qual é a consequência específica, citada no exemplo prático, da sua ausência?

A. Garantir que a equipe toda seja responsável pela Qualidade e evitar a Gestão de Conflitos. A ausência leva à dificuldade em usar o Git.
B. Gerenciar o Escopo Inicial e manter a clareza dos Objetivos do Projeto. A ausência leva ao aumento da Documentação necessária.
C. Ajudar a distribuir o trabalho de forma inteligente e definir marcos importantes. A ausência resulta em tarefas sem ordem, deixando as difíceis para o final (produto incompleto/erros).
D. Promover a Comunicação Efetiva e a Boa Dinâmica de Grupo. A ausência leva a sobrecarga do Líder de Projeto.
E. Garantir que a Origem da Ideia se mantenha relevante e que o Público-Alvo seja sempre consultado. A ausência leva ao retrabalho..

C. Ajudar a distribuir o trabalho de forma inteligente e definir marcos importantes. A ausência resulta em tarefas sem ordem, deixando as difíceis para o final (produto incompleto/erros).

GRUPOS ENG. SOFTWARE - EXERCICIOS

Em relação à Gestão de Conflitos, qual é a principal diretriz de postura estabelecida no material para guiar a resolução de desentendimentos e o que é o objetivo final dessa resolução construtiva?

A. Definir um mediador (Líder de Projeto) para intermediar o debate, tendo como objetivo a manutenção do Cronograma.
B. Garantir que a discussão seja feita em reuniões regulares, buscando como objetivo final a satisfação do cliente (Stakeholder).
C. Focar no problema, não na pessoa, buscando como objetivo o consenso e a melhor solução para o projeto.
D. Registrar a divergência na Documentação e aplicar um sistema de votação, com o objetivo de priorizar a agilidade na decisão.
E. Evitar o desentendimento ao máximo, garantindo que a equipe mantenha a Boa Dinâmica de Grupo.

C. Focar no problema, não na pessoa, buscando como objetivo o consenso e a melhor solução para o projeto

GRUPOS ENG. SOFTWARE - EXERCICIOS

A qualidade do software deve ser "construída ao longo de todo o processo", e não adicionada no final. Quem é o principal responsável pela qualidade do produto, de acordo com o texto, e qual o papel secundário atribuído aos desenvolvedores neste processo?

A. O Testador; o papel secundário dos desenvolvedores é corrigir apenas os bugs de Back-End.
B. O Líder de Projeto; o papel secundário dos desenvolvedores é implementar apenas os requisitos definidos pelo Analista.
C. A equipe toda; o papel secundário dos desenvolvedores é testar o que eles próprios codificam, em uma prática contínua.
D. O cliente (Stakeholder); o papel secundário dos desenvolvedores é realizar apenas testes automatizados.
E. O Analista de Requisitos; o papel secundário dos desenvolvedores é manter a Documentação de testes atualizada.

C. A equipe toda; o papel secundário dos desenvolvedores é testar o que eles próprios codificam, em uma prática contínua.

"A equipe toda é responsável pela qualidade do produto". Além disso, estabelece que "Desenvolvedores testam o que codificam".

GRUPOS ENG. SOFTWARE - EXERCICIOS

Em projetos colaborativos, o Sistema de Controle de Versão (VCS), como o Git, é essencial. Qual funcionalidade do Git o torna indispensável para gerenciar o risco de regressão no código-fonte, além de permitir o trabalho simultâneo e a resolução de conflitos de código?

A. Acompanhar as mudanças feitas por cada um, garantindo a rastreabilidade da autoria de cada linha de código.
B. Voltar a versões anteriores do código se algo der errado, prevenindo que um erro destrutivo se mantenha no produto.
C. Automatizar a integração contínua (CI) entre o Front-End e o Back-End, validando a lógica de negócio
D. Resolver automaticamente os conflitos entre diferentes desenvolvedores que alteraram a mesma linha de código.
E. Gerenciar a Documentação do projeto, mantendo um histórico de todas as decisões e requisitos.

B. Voltar a versões anteriores do código se algo der errado, prevenindo que um erro destrutivo se mantenha no produto.

GRUPOS ENG. SOFTWARE - EXERCICIOS

O Escopo Inicial é uma delimitação crucial. No Exemplo Prático de criação de um sistema de gerenciamento de biblioteca, o que foi definido como a **exclusão** fundamental do escopo para evitar que "o projeto se estenda indefinidamente"?

A. A criação de um módulo para consulta de livros online pelo público-alvo e o design de interface (Front-End).
B. A inclusão de um sistema de segurança robusto (Back-End) e a integração com o Sistema de Controle de Versão (VCS).
C. O desenvolvimento de funcionalidades como multas, compras de livros ou integração com sistemas financeiros.
D. A definição do Público-Alvo e dos Objetivos do Projeto mensuráveis.
E. A possibilidade de Documentação em um wiki online e a realização de testes manuais.

C. O desenvolvimento de funcionalidades como multas, compras de livros ou integração com sistemas financeiros.

GRUPOS ENG. SOFTWARE - EXERCICIOS

A Documentação é vital para a longevidade. Qual é a tríade de tipos de informação de alto nível que é essencial registrar, e qual o principal benefício dessa prática para a continuidade do projeto?

A. Histórico de bugs, código comentado e decisões do Stakeholder; o benefício é a diminuição da necessidade de reuniões regulares.
B. Prazos do Cronograma, perfis do Público-Alvo e testes realizados; o benefício é a garantia de qualidade contínua.
C. Alterações no código (VCS), arquitetura do Back-End e regras de negócio; o benefício é a automação de processos.
D. Decisões, requisitos e design do sistema; o benefício principal é ajudar novos membros e servir como base para a manutenção futura do software.
E. Conflitos resolvidos, relatórios de progresso e uso das Ferramentas de Gestão; o benefício é a rastreabilidade do trabalho em grupo.

D. Decisões, requisitos e design do sistema; o benefício principal é ajudar novos membros e servir como base para a manutenção futura do software

GRUPOS ENG. SOFTWARE - EXERCICIOS

O termo "Stakeholders" é crucial na gestão de projetos. Quem são especificamente os Stakeholders em um projeto de software?

A. Apenas os usuários finais do sistema (Público-Alvo) e os desenvolvedores que escrevem o código.
B. Pessoas ou grupos que têm interesse ou influência no projeto, como clientes, patrocinadores e usuários.
C. Os membros da equipe responsável pela Documentação e pelo Versionamento de Código (VCS).
D. O Líder de Projeto, o Analista de Requisitos e o Testador, por serem responsáveis pela qualidade.
E. Apenas os patrocinadores financeiros do projeto e a gerência sênior da organização..

B. Pessoas ou grupos que têm interesse ou influência no projeto, como clientes, patrocinadores e usuários.

GLOSSÁRIO

Termo	Definição
Back-End	Parte do sistema responsável pela lógica, regras de negócio e acesso a dados. Funciona nos bastidores, sem interface direta com o usuário.
Boa Dinâmica de Grupo	Capacidade de um grupo de trabalhar de forma colaborativa, respeitosa e produtiva, mesmo com perfis diferentes.
Comunicação Efetiva	Troca clara e constante de informações entre membros da equipe e stakeholders, essencial para evitar retrabalho e mal-entendidos.
Cronograma	Planejamento temporal que define prazos e etapas do projeto, ajudando a distribuir o trabalho ao longo do tempo.
Documentação	Registro de decisões, requisitos, design e problemas do projeto, fundamental para manutenção e continuidade.

GLOSSÁRIO

Termo	Definição
Escopo Inicial	Delimitação do que o sistema fará (ou não fará) na primeira versão. Ajuda a evitar excesso de funcionalidades.
Experiência do Usuário (UX)	Qualidade da interação do usuário com o sistema, envolvendo usabilidade, acessibilidade e satisfação geral.
Ferramentas de Gestão	Softwares como Trello, Asana e planilhas usadas para organizar tarefas, prazos e responsabilidades no projeto.
Front-End	Parte visual do sistema com a qual o usuário interage diretamente (telas, botões, formulários etc.).
Gestão de Conflitos	Prática de resolver desentendimentos no grupo de forma construtiva, sem prejudicar o andamento do projeto.

GLOSSÁRIO

Termo	Definição
Git	Sistema de controle de versão amplamente usado no desenvolvimento de software para rastrear e gerenciar alterações no código.
GitHub/GitLab	Plataformas baseadas em Git que permitem colaboração em projetos, controle de versões e integração com ferramentas de desenvolvimento.
Líder de Projeto	Pessoa responsável por coordenar tarefas, organizar reuniões e garantir que o grupo siga o plano de projeto.
Objetivos do Projeto	Resultados específicos que se espera alcançar com o software, definidos no início do projeto.
Público-Alvo	Usuários finais do sistema. Conhecer o público ajuda a guiar decisões de design, funcionalidade e linguagem.
Reuniões Regulares	Encontros planejados entre membros da equipe para alinhar tarefas, discutir progresso e resolver impedimentos.

GLOSSÁRIO

Termo	Definição
Stakeholders	Pessoas ou grupos que têm interesse ou influência no projeto, como clientes, patrocinadores e usuários.
Sistema de Controle de Versão (VCS)	Ferramenta usada para registrar alterações no código-fonte de forma organizada, permitindo trabalho colaborativo seguro.
Testes	Processo de verificação de que o software funciona conforme os requisitos. Inclui testes automatizados e manuais.
Trabalho em Grupo	Colaboração entre várias pessoas com habilidades diversas para alcançar um objetivo comum no desenvolvimento de software.

BIBLIOGRAFIA BÁSICA

- PAULA FILHO, Wilson de Pádua. Engenharia de Software: Fundamentos, Métodos e Padrões. LTC, 2009.
- PRESSMAN, R. S. Engenharia de software. 6.ed. McGraw-Hill, 2006.
- SOMMERVILLE, I. Engenharia de software. 8.ed. Addison Wesley, 2007



OBRIGADO

ARMANDO VULCANO JR

ARMANDO.FATEC@GMAIL.COM